CS/CE 224/227: Object Oriented Programming and Design Methodologies

# Week 11/Unit 11: Virtual Functions

Tauqeer Saleem

Slides based on Dr Faisal Alvi's slides

For any suggested modifications please email: tauqeer.saleem@sse.habib.edu.pk

# Unit Outline

1. Splitting Code into files
2. Virtual Functions
3. Pure Virtual Functions and Abstract Classes
4. Summary

# Reminder of Splitting Project into header and source files

# Splitting Code Into Files

- When you write and compile small programs, you can place all your code into a single source file.
- When your programs get larger or you work in a team, you will want to split your code into separate source files.
- Reason#1 It takes time to compile a file, and it is not efficient to wait for the compiler to keep translating code that doesn't change.
- Reason#2 When you work with other programmers in a team, it would be very difficult for multiple programmers to edit a single source file simultaneously.

# The Header File

- If your program is composed of multiple files, some of these files will define data types or functions that are needed in other files.
- There must be a path of communication between the files.
- In C++, that communication happens through the inclusion of header files.
- Header file: A file that informs the compiler of features that are available in another module or library.

# Header and Source Files

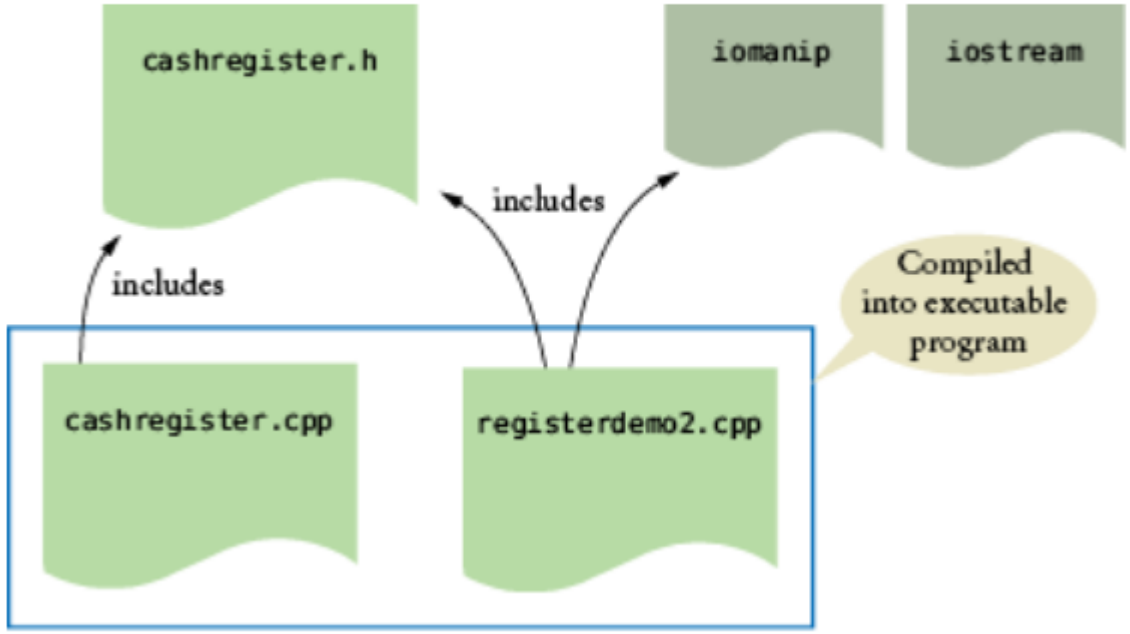
- A header (.h) file contains
  - Definitions of classes.
  - Definitions of constants.
  - Declarations of nonmember functions.
- The source (.cpp) file contains
  - Definitions of member functions.
  - Definitions of nonmember functions.

  - Example: CashRegister file (Chapter 9 of Horstmann)

# CashRegister Example (Snippets)

```
#ifndef CASHREGISTER_H
#define CASHREGISTER_H
. . .
#endif
```

Figure 9.15.1: Compiling a Program from Multiple Source Files.



# Slicing Problem

# The Slicing Problem

- In object oriented programming, a derived class object can be assigned to a base class type.
- In other words, the assignment `Shape s = Circle c;` is valid.
- When a derived class object is assigned to a base class object in C++, the derived class object's extra attributes are sliced off (not considered) to generate the base class object.
- This whole process is termed **object slicing**.

# Virtual Functions

# Virtual Functions

- One application of inheritance is to work with objects whose type and behavior can vary at run time.
- This variation of behavior is achieved with *virtual functions*.
- When you invoke a virtual function on an object, the C++ run-time system determines which actual member function to call, depending on the class to which the object belongs.
- Definition of virtual function: *A function that can be redefined in a derived class. The actual function called depends on the type of the object on which it is invoked at run time.*

# Virtual Functions

- The `virtual` reserved word must be used in the base class.
- All functions with the same name and parameter variable types in derived classes are then automatically virtual.
- However, it is considered good practice to supply the `virtual` reserved word for the derived-class functions as well.
- The keyword ‘override’ is also used *after* the declaration of the virtual function.

# Virtual Functions

- In order to call the functions from the **actual object type**, rather than the **reference type**, we declare each such function

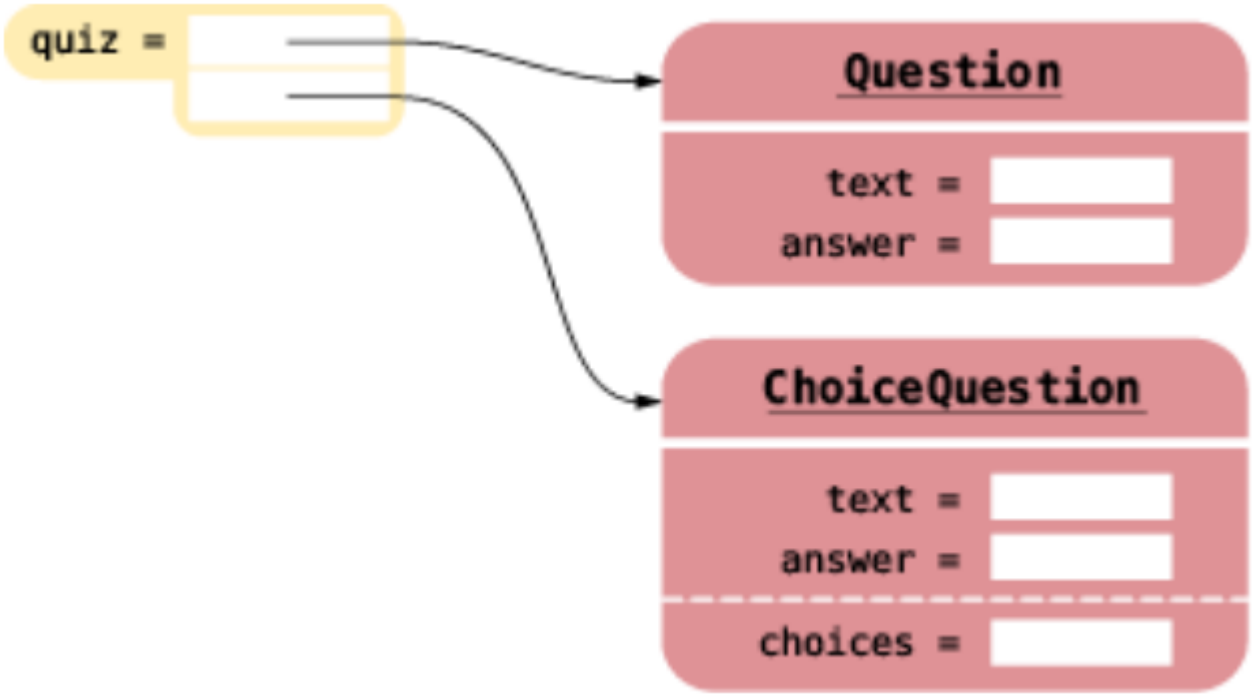
```
class Question
{

public:
   Question();
   void set_text(string question_text);
   void set_answer(string correct_response);
   virtual bool check_answer(string response) const;
   virtual void display() const;
private:
   . . .
};
```

# A Collection of Objects

- Suppose we have a collection of objects: some from the base class, others from the derived classes.
- We wish to put these into a collection of objects, such as an array.
- However, derived-class objects are usually bigger than base-class objects, and objects of different derived classes have different sizes.
- An array of objects cannot deal with this variation in sizes.
- Instead, we store the actual objects elsewhere and collect their locations in an array by storing pointers.

# An Array of Pointers (to Objects)

Figure 10.5.2: An Array of Pointers Can Store Objects from Different Classes.



# An Array of Pointers

Here is the code to set up the array of pointers (see Figure 10.5.2):

```
Question* quiz[2];
quiz[0] = new Question;
quiz[0]->set_text("Who was the inventor of C++?");
quiz[0]->set_answer("Bjarne Stroustrup");
ChoiceQuestion* cq_pointer = new ChoiceQuestion;
cq_pointer->set_text("In which country was the inventor of C++ born?");
cq_pointer->add_choice("Australia", false);
. . .
quiz[1] = cq_pointer;
```

Here **reference type** is `Question`, but **actual object type** is `ChoiceQuestion`

# Polymorphism

# Polymorphism – calling functions from the correct object type

- We have a collection of various question objects in our array of quiz.
- We wish to display questions, such that the corresponding `display` function is called:

```
for (int i = 0; i < QUIZZES; i++)
{
   quiz[i]->display();
   cout << "Your answer: ";
   getline(cin, response);
   cout << quiz[i]->check_answer(response) << endl;
}
```

- Likewise, we would like to call the correct `check_answer` function.

# Polymorphism

- The quiz array collects a mixture of both kinds of questions.
- Such a collection is called polymorphic (literally, “of multiple shapes”).
- Objects in a polymorphic collection have some commonality but are not necessarily of the same type.
- Inheritance is used to express this commonality, and virtual functions enable variations in behavior.
- (Definition of polymorphism: Selecting a function among several functions that have the same name on the basis of the actual type of the implicit parameter.)

# Virtual Functions - Advantages

- Virtual functions give programs a great deal of flexibility.
- The question presentation loop describes only the general mechanism: “Display the question, get a response, and check it”.
- Each object knows on its own how to carry out the specific tasks: “Display the question” and “Check a response”.
- Using virtual functions makes programs easily extensible.
- One or more new classes may be added such as `NumericQuestion`.
- All that is needed is to override virtual functions in the new classes with the correct implementation.

# Virtual Self Calls

- Consider the following code snippet:

```
void Question::ask() const
{
    display();
    cout << "Your answer: ";
    getline(cin, response);
    cout << check_answer(response) << endl;
}
```

- This function is in the `Question` class – and inherited by all other derived classes.

# Virtual Self Calls

Now consider the call

```
ChoiceQuestion cq;
cq.set_text("In which country was the inventor of
C++ born?");
. . .
cq.ask();
```

Which `display` and `check_answer` function will the `ask` function call?

# Virtual Destructors

- Base class destructors should always be virtual.
- Suppose you use delete with a base class pointer to a derived class object to destroy the derived-class object.
- If the base-class destructor is not virtual then delete, like a normal member function, calls the destructor for the base class, not the destructor for the derived class.
- This will cause only the base part of the object to be destroyed.

# Abstract Classes

# Pure Virtual Functions

- A “pure” virtual function does not have an implementation. (*exceptions?)
- A “pure” virtual function is abstract – i.e. it exists only in name.
- From a syntactical point of view, a pure virtual function is declared by putting the keywords =0 after the function declaration.
- Example:

```
class Base                              //base class
   {
   public:
      virtual void show() = 0;          //pure virtual function
   };
```

# Pure Virtual Functions

Here the virtual function `show()` is declared as

```
virtual void show() = 0;  // pure virtual function
```

The equal sign here has nothing to do with assignment; the value 0 is not assigned to anything. The `=0` syntax is simply how we tell the compiler that a virtual function will be pure.

# Abstract Classes

- A class containing a pure virtual function is an *abstract class.*
- An abstract class is not instantiated – i.e. objects cannot be created out of an abstract class.
- An abstract serves more like an interface or a template for derived classes.
- A derived class inherits all of the functions of the base class – including pure virtual functions.
- If a derived class implements pure virtual functions by overriding them, then objects of the derived class can be instantiated.
- Otherwise the derived class is also *abstract.*

# Examples

# Example: Pure Virtual Functions

```
//////////////////////////////////////////////////////////////
class person                                  //person class
   {
   protected:
      char name[40];
   public:
      void getName()
         { cout << "    Enter name: "; cin >> name; }
      void putName()
         { cout << "Name is: " << name << endl; }
      virtual void getData() = 0;             //pure virtual func
      virtual bool isOutstanding() = 0;       //pure virtual func
   };
```

# Example: Pure Virtual Functions

```
class student : public person          //student class
   {
   private:
      float gpa;                  //grade point average
   public:
      void getData()              //get student data from user
         {
         person::getName();
         cout << "    Enter student's GPA: "; cin >> gpa;
         }
      bool isOutstanding()
         { return (gpa > 3.5) ? true : false; }
   };
```

# Example: Pure Virtual Functions

```
class professor : public person      //professor class
   {
   private:
      int numPubs;                   //number of papers published
   public:
      void getData()                 //get professor data from user
         {
         person::getName();
         cout << "   Enter number of professor's publications: ";
         cin >> numPubs;
         }
      bool isOutstanding()
         { return (numPubs > 10 ) ? true : false; }
   };
```

# Example: Pure Virtual Functions

```
int main()
   {
   person* persPtr[100];      //array of pointers to persons
   int n = 0;                 //number of persons on list
   char choice;

   do {
      cout << "Enter student or professor (s/p): ";
      cin >> choice;
      if(choice=='s')                     //put new student
         persPtr[n] = new student;        //   in array
      else                                //put new professor
         persPtr[n] = new professor;      //   in array
      persPtr[n++]->getData();            //get data for person
      cout << "   Enter another (y/n)? ";  //do another person?
      cin >> choice;
      } while( choice=='y' );             //cycle until not 'y'
```

# Example: Pure Virtual Functions

```
for(int j=0; j<n; j++)                    //print names of all
   {                                      //persons, and
   persPtr[j]->putName();                 //say if outstanding
   if( persPtr[j]->isOutstanding() )
      cout << "   This person is outstanding\n";
   }
return 0;
}  //end main()
```

# Virtual Functions - Summary

- Virtual Functions are a useful mechanism in C++ to implement polymorphism.
- Virtual Functions begin with the keyword ‘virtual’.
- A derived class is supposed to override a virtual function inherited from the base class to exhibit polymorphic behavior.
- Pure Virtual Functions do not have any implementation.
- A class containing a pure virtual function is abstract.